

Take-Home Assignment: GTS Poland hiring

Context

For this assignment, you'll build a backend service for managing employee profiles. Part 1 focuses on creating a well-designed CRUD API. Part 2 extends it with a bulk import feature.

You are encouraged to use AI-assisted coding tools (Copilot, Claude, Cursor, ChatGPT, Kiro, etc.) throughout this exercise, that's expected and normal. What we care about is the engineering judgment behind the result: what you chose to keep, what you changed, and why.

Since AI tools can generate syntactically correct code, we're primarily evaluating your logical thinking, decision-making, and ability to reason about trade-offs — not raw coding ability. The code is the vehicle; your judgment is what we're assessing.

Part 1: Creating CRUD operation for Employee Profile (Primary Focus)

Assignment: Employee Profile Management System

Objective: Design and develop an API for an Employee Profile Management System that handles the creation, retrieval, update, and deletion of employee records.

Requirements:

- Programming Language – Build the API using a language/framework of your choice (e.g., Python, Node.js, Java, or any other suitable technology).
- Database – Integrate a database of your choice to persist employee data (e.g., PostgreSQL, MongoDB, MySQL, etc.).
- Technology Justification – Provide a clear explanation of why you chose your specific technology stack (language + database), including the reasoning and trade-offs considered in your decision.
- See Part 2 for the full API specification — implement all endpoints listed there

Data Model — each Employee record should include:

- id (system-generated, unique)
- Name
- Hiredate
- Email
- PhoneNo
- ProfilePicture (store as a URL string)
- Status(active,inactive)
- Address
- State
- Country
- City
- Pincode
- CreatedAt

Note: Use this model for both parts.

This is just a Sample page of how the data appears on UI. This is for reference only — no UI is required

Employee's Information

Add New Employee

Search

Name	Department	Address	City	Country	Action
Nisha	adad	Hyderabad	Hyderabad	India	EditDelete
Sanwar	IT	C-Scheme	Jaipur	India	EditDelete
Nisha	IT	C-Scheme	Jaipur	India	EditDelete
Sanwar Ranwa	IT	C-Schme	Jaipur	India	EditDelete
Sanwar Ranwa	IT	C-Schme	Jaipur	India	EditDelete

< Previous

12

Next >

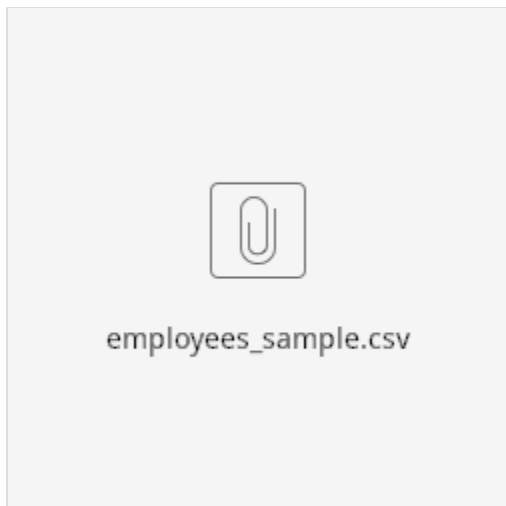
When building your API, avoid these common pitfalls:

- Security — SQL injection, unsafe ID generation, input trust

- **Error handling** — unhandled async/await, missing try/catch, no checks for missing records
 - **Correctness** — off-by-one errors, logic bugs, weak validation (e.g., `== ''` vs proper null/undefined checks)
 - **Business rules** — date not validated against current date, invalid status silently corrected instead of rejected
 - **API design** — wrong HTTP status codes, GET endpoint triggering side effects (sending notifications)
 - **Code quality** — `var` vs `let/const`, inconsistent async handling, missing `await` on DB calls
 - **Robustness** — no handling for external service failures, no input validation on update, no 404 responses
-

Part 2: Extend the Service (Secondary)

Extend the above with a working implementation and build a bulk load option to import a bunch of employees from a csv file. Attached is the csv file.



Requirements

Data Model — See Part 1 for the full data model and use it for both parts.

Business Rules:

- Email must be unique across all employee records
- HireDate cannot be a future date
- Phone number must follow a valid format (e.g., E.164 or country-specific)
- Add 3-4 additional business rules, validations and explain each of them
- Use the given CSV to load the data using the bulk API.

REST API:

- POST /employee - create a new employee
- GET /employee/{id} - retrieve a single employee
- GET /employees - list all employees
- PUT /employee/{id} - update employee
- DELETE /employee/{id} - delete employee
- POST /employees/bulk - import employees from a CSV file

Persistence:

- Use a real database (relational or non-relational — your choice).
- Include whatever is needed to run it locally (e.g., Docker Compose, embedded DB, or clear setup instructions).

Validation & Error Handling:

- Validate required fields, date format, and allowed status values.
- Return clear error responses with appropriate HTTP status codes (400, 404, etc.).

Tests:

- Include unit tests covering at least:
 - employee validation logic
 - employee update logic
 - At least one API endpoint
-

What We're Assessing

- **Security awareness** — Do you recognize injection vulnerabilities and unsafe patterns?
 - **Code quality and reasoning** — Is your code clean, and why you made each choice
 - **Implementation fundamentals** — Can you build a simple, correct service from the ground up?
 - **Testing judgment** — What did you choose to test and why?
 - **AI tool usage** — What you accepted, questioned, or rewrote from AI suggestions.
-

Deliverables

A public GitHub repository containing:

- Your employee Management service (working code)
 - Code should be clean and readable
 - [README.md](#) covering:
 - How to run the project locally
 - Your technology choices and why
 - API examples or sample requests
 - Anything you'd do differently with more time
 - A brief note on your AI tool usage: which tools you used, what you used them for, and one example of something you changed or rejected from an AI suggestion
 - Provide a one-to-two sentence explanation for each major design decision made during implementation
-

Time Expectation

This is designed to take a few focused hours. We're not expecting a production-grade system, just clear thinking and working code.

Interview Follow-up

Be prepared to walk us through your code and explain:

- Which aspects you considered most critical and why
- Your approach to preventing injection vulnerabilities
- Your API design and validation approach
- Your database choice
- Testing strategy
- Your approach to handling failures in the bulk import process.
- How AI tools influenced your implementation and decision-making